

Masks for Skipper-CCD images that do not depend on the sensor

Can the masks that remove readout defects, material defects and unwanted physics be written so that they work on a sensor they were never tuned on?

D. Rodríguez Juiz and F. Pérez · final project, *Gravitational Waves and AI-Assisted Research* · full report, code and provenance: <https://delfinarj.github.io/GW-AI-final-project/>

Short answer. Yes for five of the six masks, and the price is measurable. Of the 28 cases in which a mask tuned by hand on one sensor was moved unchanged to another, 5 were worse than using no mask at all in every seed and 2 made no difference; the clearest, low-energy clusters on the surface laboratory sensor (tuned on deep underground), scores 0.12 of the mask tuned with truth on that sensor, where doing nothing scores 1.00. The same masks written as self-calibrating procedures are worse than no mask in 1 of 14 cases and better in 12, and reach a median 0.98 of the mask tuned with truth. The exception is low-energy clusters on the surface laboratory sensor (adaptive), which fires on charge-transfer trail electrons.

The question and what was built

A Skipper-CCD counts single electrons. Before any physics is extracted, analyses discard pixels with masks, each aimed at one problem: charge left behind during transfer, hits in the serial register, hot columns and pixels, photons around high-energy tracks (the halo), clusters of low-energy events, and muon tracks. Every mask carries sizes and thresholds — a radius, a trail length, a rate cut — chosen by hand for one sensor and site.

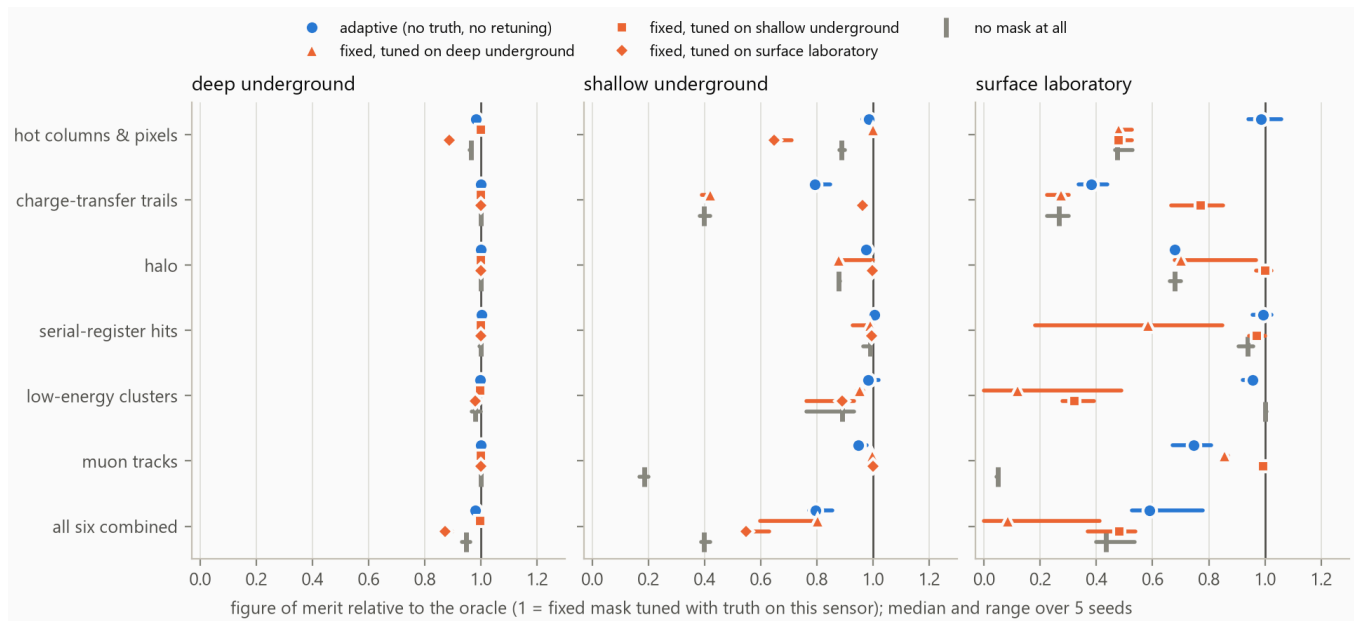
Hypothesis: what transfers between sensors is not the numbers but the *procedure* that chooses them. Each of the six masks was therefore written twice: a **fixed** form with hand-set constants, and an **adaptive** form that measures its own sizes on the images it is applied to and sets its threshold from a false-positive rate ($\alpha = 0.01$, corrected for the number of tests) or from measurable physical properties of the sensor.

The test bed is a simulator that keeps a separate charge map per source, so a mask can be scored against exactly the events it should remove. Three sensors differ the way real deployments do; their geometry, noise, dark rates, backgrounds and muon fluxes come from public measurements, while defect populations, which are not published in a transferable form, are scenario values chosen to differ between sensors.

Sensor	Pixels	Thickness (μm)	Noise (e^-)	Exposure	Dark ($\text{e}^-/\text{pix}/\text{day}$)	Muons ($\text{cm}^{-2}/\text{day}$)
Deep underground	3072×512	665	0.14	20 h	1.4×10^{-5}	2.9e-05
Shallow underground	3072×443	675	0.14	20 h	1.6×10^{-4}	6.9
Surface laboratory	1278×1058	725	0.17	1 h	3.0×10^{-2}	1.4e+03

Four forms of every mask were compared on an independent test stack of each sensor: the **oracle** (fixed, tuned with the simulator's truth on that sensor), the **transplant** (the oracle constants of a different sensor), the **adaptive** form, and **no mask**. The figure of merit is $S/\sqrt{S+B}$: surviving injected signal against surviving target background.

Result 1 · Transplanted constants can do harm; self-calibration avoids the large failures, at a cost



Each mask on each sensor, relative to the oracle of that sensor, over 5 seeds. A transplanted mask is the oracle of another sensor; no mask is the reference that says whether masking helps at all.

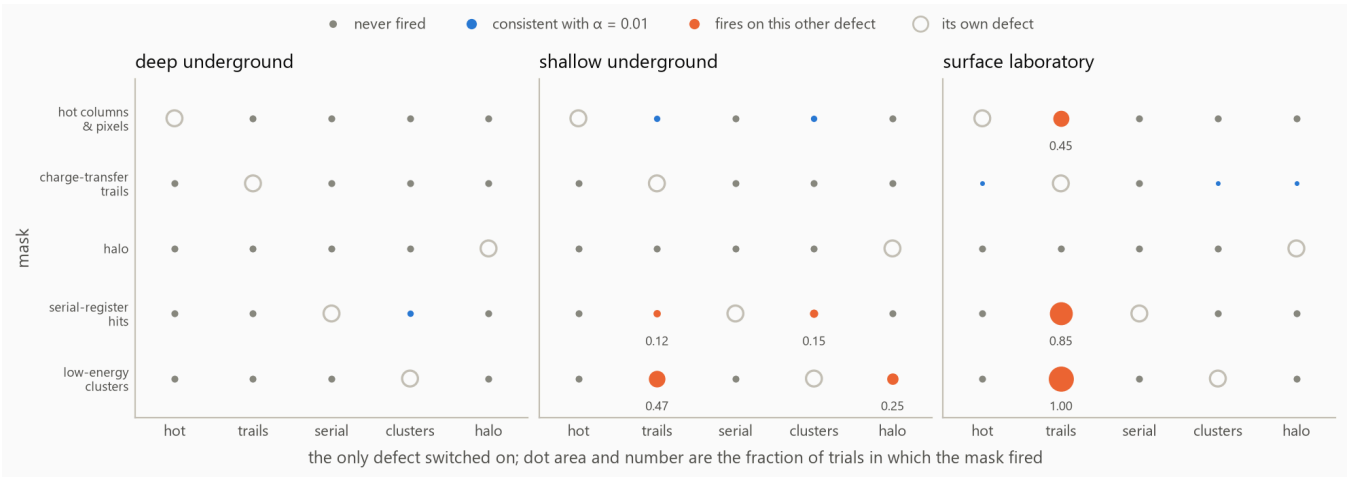
Moving constants between sensors is not merely suboptimal, it can be worse than not masking: 5 of 28 transplants score below no mask in every seed, by more than the 1 % margin used to call a difference real. Self-calibration removes those failures — 1 of 14 cases — but it is not free: in 8 cases a transplanted mask still beats the adaptive one in every seed, and the worst adaptive case, charge-transfer trails on the surface laboratory sensor (adaptive), reaches only 0.38 of the oracle (0.34 in its worst seed).

The summary statistics were pre-registered before the final run. Cases with fewer than 20 target events in a seed are excluded from that seed, and a case counts as harmful only if it loses by more than the margin in *every* valid seed. Two seeds (2) were run once after the code was frozen and never used to change anything: on them, 4 of 26 transplants are harmful, 0 of 13 adaptive cases are, and the adaptive median is 0.98 of the oracle — the same picture as the development seeds.

Result 2 · What the masks fire on when there is nothing to find

With every defect switched off on all three sensors, keeping what a real sensor cannot switch off (dark current, spurious charge, the injected signal, muon tracks and high-energy deposits), the five statistical masks fired 5 times in 15 sensor-mask tests of 50 runs each, and every interval contains α . The sixth mask, for muon tracks, cannot be in that test, since tracks are what a real sensor cannot switch off; asked separately, in a simulator with the flux set to zero, it fired in 0 of 600 images.

That test has a blind spot: switching every defect off at once cannot see a mask firing on *another* mask's defect. So the defects were also switched on one at a time, 60 mask-and-defect cells in all, 20 runs each, with the bound each cell must clear taken at 99.92 % confidence to control a 5 % chance of one false call over the whole grid.



One defect on at a time. Dot area is the fraction of trials in which the mask fired; orange marks the cells above the rate that mask should not exceed; hollow circles are the diagonal, where a mask meets its own defect.

7 of 60 cells fire on a defect that is not their own — and the 6 of them counted per image survive being recounted one run at a time, which is the conservative way to treat two images that share a calibration. Charge-transfer trails are what most often set them off. The largest is unambiguous: on the surface sensor, with trails as the only defect present, the adaptive low-energy-cluster mask fires in every one of its trials and masks a median 0.121 of the image. This is the one real failure of self-calibration found here, and it was predicted in writing before the run, from a symptom seen in Result 1.

Sensor	Only defect present	Mask that fired	Fired/trials	Rate	Median fraction masked
Surface laboratory	charge-transfer trails	low-energy clusters	40/40	1.00	0.1215
Surface laboratory	charge-transfer trails	serial-register hits	34/40	0.85	0.0009
Shallow underground	charge-transfer trails	low-energy clusters	19/40	0.47	0.0000
Surface laboratory	charge-transfer trails	hot columns and pixels	9/20	0.45	0.0000
Shallow underground	halo	low-energy clusters	10/40	0.25	0.0000
Shallow underground	low-energy clusters	serial-register hits	6/40	0.15	0.0000
Shallow underground	charge-transfer trails	serial-register hits	5/40	0.12	0.0000

Result 3 · The same procedures on a real sensor

Everything above is simulated. The adaptive masks were also run, unchanged, on the 4 exposures of the public SENSEI SNOLAB data release: a fourth sensor, real, with a geometry none of the presets has (3072×16 active superpixels, each binning 32 physical rows). Nothing about it was tuned, and the release publishes its own mask, so the two can be compared where they overlap.

Given only the images, the estimator measures a readout noise of 0.141–0.142 e, against 0.1413–0.1420 e from an independent fit on the same files, and a single-electron density that grows with exposure. The hot-column procedure flags 2, 4, 6, 7 columns as the exposure grows, and the sets are nested: each keeps the previous columns and adds the next loudest. At the longest exposure the 7 columns with the highest rate of charged pixels are exactly the ones it flags, all of them inside the release's own bad-column mask, and each carries 102 to 1830 times the charged-pixel rate of the columns it left alone.

Exposure	Images	Trigger pixels	Noise (e ⁻)	1e density	Hot columns	Halo radius	We mask	The release masks
0 h	14	0	0.142	1.3×10^{-4}	2	0	0.0007	0.042
2 h	14	70	0.142	1.7×10^{-4}	4	0	0.0013	0.055
6 h	14	103	0.141	2.9×10^{-4}	6	0	0.0038	0.061
20 h	13	844	0.141	6.5×10^{-4}	7	25	0.0094	0.130

Both directions of the agreement matter and both are here. Of what the procedure flags, the release also flags 0.99–1.00: it stays inside a mask a person tuned. Of what the release flags, the procedure flags 0.027–0.095: the release masks far more. The first number is the easy direction — a procedure that flagged one true column would score 1.00 — and it rests on a handful of column decisions, not on the hundreds of pixels it is counted over.

The trail calibration chose length zero on all four exposures, which the registered expectation did not predict. The reason is in the result file: the release blinds its hits, so a whole exposure holds 0, 70, 103, 844 pixels above the trigger. At the longest exposure the test does see the excess on the correct side — 13 single electrons downstream of a trigger against 2 upstream, $p = 0.0037$ — but the corrected threshold asks for $p < 6.3e-05$. The procedure is refusing to mask on evidence this thin, which is what it was built to do.

How the work was done, and how it was checked

Every line of code and every analysis in this project was produced by AI agents, working from a stated question and under rules written down before the work: no number on any page is typed by hand, every output carries a sidecar recording the script, the git commit, the hashes of its inputs and its seed, and no private document ever enters a tracked file. The method that made this trustworthy, rather than merely fast, was adversarial:

- **Expectations were registered before runs.** Written in `PLAN.md` and committed before the analysis ran, so a confirmation could not be invented afterwards. One of them was wrong — the trail length on the real sensor — and the repository says so, with the numbers that explain why.
- **Independent agents reviewed the work** with no access to the conversation that produced it. They found real defects: truth from the simulator leaking into an "adaptive" threshold; a headline that measured doing nothing; a halo mask measuring distance in superpixels on a binned sensor, which hid the loudest column from the hot-column calibration; a verdict applied to sixty cells with no correction for their number. Each is recorded in `PROVENANCE.md` with what was done about it, including the findings that were *not* acted on.
- **Every result was reproduced from a clean clone** — a fresh checkout sharing nothing with the working copy but the commit and the pinned environment — and compared at a relative tolerance of 10^{-9} . All of them came back bit-identical, figures included. The slowest, two and a half hours, was resumed after something killed it part way through, which is what showed that the command a stranger would run to reproduce it did nothing at all in a fresh clone: it found the finished file we ship and skipped the work.

Errors the checks caught are listed on the full report page rather than hidden: a simulator that redrew its hot columns every image, a hot-column mask that flagged innocent neighbours, a halo test that treated its reference rate as exact, a muon mask that missed crossing tracks, a calibration order that made 69 of 81 flagged columns false.

What is still wrong with it

- Defect populations are scenario values, not measurements; only the sensor and background parameters are public measurements.
- The defects are switched on one at a time, so nothing here measures what the masks do when several defects overlap in the same pixels.
- On the real sensor only four of the six masks can be checked at all: the release publishes no counterpart to the low-energy-cluster mask, and its binned superpixels make the muon mask's geometry meaningless.
- One figure of merit, and an oracle tuned mask by mask rather than jointly, so a combined adaptive mask can exceed it.

Where everything is

Repository: <https://github.com/delfinarj/GW-AI-final-project> — `README.md` reproduces the whole project from a clean clone with `uv`; `PLAN.md` holds the pre-registered statements; `PROVENANCE.md` records every input, every choice with a defensible alternative, how each result was checked, and every error found along the way; `report/index.html` is the full report, of which this document is a summary.